PulseGrid

Real-Time Energy Market Intelligence Platform

An end-to-end medallion lakehouse on Microsoft Fabric — ingesting live electricity prices from 40 European and US market zones, predicting price spikes with XGBoost, explaining every prediction with SHAP, and serving it through Power BI and a Claude-powered analyst.

MICROSOFT FABRIC · PYSPARK · KQL · DELTA · XGBOOST · MLFLOW · SHAP · POWER BI · STREAMLIT · CLAUDE

LIVE APPLICATION
pulsegrid-energy.streamlit.app

SOURCE
github.com/demonjd2026-afk/pulsegrid-fabric-realtime

AUTHOR
Jayanth Dolai

Electricity prices move fast.
Knowing why is the hard part.

European day-ahead power markets clear hourly across 27 bidding zones. Prices swing from negative during renewable oversupply to several hundred euros per MWh in a demand peak — often within the same trading day. Traders, grid operators and industrial buyers all need the same two answers, quickly.

Will prices spike in the next 2 hours?

A spike is defined as the price exceeding the zone's own 90th percentile. Predicting it means combining price history, grid load, generation mix, cross-border flows and weather — five different data sources on five different publication cadences.

And *why* is it about to spike?

A probability with no explanation is not actionable. Every prediction needs its contributing factors ranked and signed, so an operator can see whether the driver is the 6-hour trend, yesterday's same hour, or the load on the wire right now.

PulseGrid is the platform that answers both — continuously, on live public data, built end to end on Microsoft Fabric Trial with no paid services, and deployed to a public URL anyone can open.

The platform in twelve numbers

4
PUBLIC APIS
ENTSO-E, EIA, Visual Crossing

40
MARKET ZONES
27 European + 13 US

6
NOTEBOOKS
ingest → silver → gold → ML

6
FABRIC PIPELINES
all scheduled, all monitored

16
TABLES BUILT
5 bronze · 5 silver · 6 gold

14
MODEL FEATURES
lags, rolling, calendar, weather

7,700
SHAP RECORDS
one row per feature per prediction

550
SCORED PREDICTIONS
across 40 zones

72.3%
MODEL ACCURACY
latest production run

73.6%
PRECISION
right 3 times in 4 when it alerts

2
SERVING SURFACES
Power BI + public web app

0
PAID SERVICES
Fabric Trial + free API tiers

Medallion lakehouse, end to end

SOURCES
Public APIs

ENTSO-E — prices, load, generation, flows
EIA — US RTO demand
Visual Crossing — weather, 20 cities

›

BRONZE
KQL Database

5 append-only tables
Schema-on-read
90-day retention
3 event-aligned pollers

›

SILVER
Lakehouse Delta

5 cleansed tables
Parallel PySpark
Dedup + schema enforce
Idempotent MERGE

›

GOLD
Curated + ML

6 serving tables
Window features
Predictions + SHAP
OPTIMIZE + ZORDER

›

SERVING
Two surfaces

Power BI Direct Lake
Streamlit web app
Claude analyst
Public URL

Ingestion is event-aligned

Each source is polled at its own publication frequency — day-ahead prices once daily, grid load every 15 minutes, weather every 30. Nothing is re-fetched just because a scheduler ticked.

Every write is idempotent

Silver and Gold both write through a MERGE on the natural key. A pipeline retry, a manual rerun, or an overlapping window all converge to the same table state.

Serving has no auth dependency

Fabric bearer tokens expire hourly. The Gold tables the web app reads are published to Git as JSON on every Gold run, so the deployed app never authenticates.

What each layer is actually built on

LAYER	TECHNOLOGY
Platform	Microsoft Fabric Trial — `PulseGrid` workspace
Real-time store	KQL Database in an Eventhouse
Lakehouse	OneLake Delta — Silver + Gold
Transformations	PySpark in Fabric Notebooks
Secrets	Fabric Environment Spark properties
ML	XGBoost · MLflow · SHAP
Orchestration	Fabric Data Pipelines — 6 schedules
Monitoring	Fabric Activator — KQL freshness rule

SURFACE	TECHNOLOGY
BI	Power BI semantic model, Direct Lake
Web app	Streamlit 1.40 · Plotly 5.20
AI analyst	Anthropic Claude API, streaming
Snapshot transport	GitHub contents API from Spark
Hosting	Streamlit Community Cloud
Data format	Delta Lake · JSON snapshot
Query languages	KQL · Spark SQL · DAX
Runtime	Python 3.10 on Fabric Spark

Constraint that shaped the stack. Fabric Trial has no Data Agent, blocks outbound calls to `api.open-meteo.com` from Spark executors, and returns a 430 capacity error when Spark sessions overlap. Each of those pushed a design decision: Claude replaces the Data Agent, Visual Crossing replaces Open-Meteo, and every pipeline runs with max concurrency 1.

Four public APIs, three pollers, zero rate-limit breaches

SOURCE	DATA	COVERAGE	CADENCE	HEADROOM
ENTSO-E	Day-ahead electricity prices	27 bidding zones	`daily 13:00`	99% of 400 req/min
ENTSO-E	Load, generation mix, cross-border flows	27 zones, 15 borders	`every 15 min`	99% of 400 req/min
EIA Open Data	RTO electricity demand	13 US authorities	`every 30 min`	conservative
Visual Crossing	Temperature, wind, humidity, solar	20 cities	`every 30 min`	960 of 1,000/day

Event-aligned polling

Each source runs on its own pipeline at its own publication frequency — the single biggest lever on rate-limit budget.

Backoff with jitter

Every call retries on `2^attempt + jitter`. Missing-data errors skip immediately — no point retrying data that was never published.

Two-stage fetch

The realtime poller tries a narrow 30-minute window first, then falls back to a 6-hour window for TSOs that publish late.

Secrets never in code

All three API keys plus the GitHub PAT live as Spark properties in the `pulsegrid_env` Environment.

A real blocker, solved. Open-Meteo was the planned weather source — 10,000 free calls a day. Fabric Trial Spark executors block outbound HTTP to it entirely. Diagnosed with a connectivity probe cell, then swapped for Visual Crossing, which is reachable and stays inside the free tier at 960 of 1,000 daily records.

Sixteen tables across three refinement stages

BRONZE — KQL Database

Append-only, schema-on-read, no transformations. 90-day retention, recoverability disabled for Trial storage limits.

```
raw_electricity_prices
raw_electricity_load
raw_generation_mix
raw_cross_border_flows
raw_weather
```

SILVER — Lakehouse Delta

All five tables cleansed *concurrently* through a ThreadPoolExecutor — wall time tracks the slowest table, not the sum of five.

```
silver_electricity_prices
silver_electricity_load
silver_generation_mix
silver_cross_border_flows
silver_weather
```

GOLD — Curated + ML

Feature engineering, aggregates, model output and explanations — the serving contract for both Power BI and the web app.

```
gold_price_features
gold_generation_summary
gold_flow_summary
gold_price_aggregates
gold_price_predictions
gold_shap_values
```

Silver cleansing rules — physical plausibility, not just types

Prices outside –500 to 5,000 EUR/MWh are nulled, but negative prices are kept — European markets clear negative during renewable oversupply. Load ≤ 0 is impossible and nulled. Self-flows where source equals destination are removed as data errors. Negative cross-border flow is kept — it means import direction. Deduplication uses `row_number()` over each table's natural key.

Gold features — computed with window functions, never collected

Price lags at 1h, 12h and 24h, a 6-hour rolling mean and standard deviation via a seconds-based `rangeBetween` window, the p90 spike threshold via `percentile_approx()`, and a broadcast join against a holidays reference table. Nothing reaches the driver until the model boundary.

Twelve Spark optimizations, each with a reason

TECHNIQUE	WHY
Predicate pushdown into KQL	Kusto trims before Spark reads
Native functions, no UDFs	Catalyst-visible, no serialisation
Window dedup `row_number()`	Distributed, deterministic
ThreadPoolExecutor, 5 workers	Wall time = slowest table
`repartition()` by region + hour	Aligns to Gold access pattern
`partitionBy(y,m,d)`	Avoids small-files problem

TECHNIQUE	WHY
`broadcast()` on holidays	16-row table, kills the shuffle
Window functions for lags	No `collect()` to driver
Adaptive Query Execution	Coalesces post-shuffle partitions
`cache()` / `persist()`	Price frame is read repeatedly
OPTIMIZE + ZORDER	File skipping on region + time
`toPandas()` at model boundary only	Avoids driver OOM at scale

The parallelism decision. The five Bronze tables are fully independent, so the cleansing notebook submits all five to a ThreadPoolExecutor rather than looping. Execution order in the log is decided by Spark job completion, not submission order — which is the visible proof it is genuinely concurrent.

The boundary decision. Null handling and casting stay in Spark using `F.coalesce()`; only the final clean frame crosses into pandas for XGBoost. That keeps the driver out of the memory path for everything except the model fit itself.

The spike predictor — XGBoost, tracked and explained

ATTRIBUTE	VALUE
Problem	Binary — price > zone p90 within 2h
Algorithm	XGBoost classifier
Estimators / depth	`100 / 4`
Learning rate	`0.1`
Min child weight	`5`
Class balance	auto `scale_pos_weight`
Eval metric	logloss
Split	time-aware, 80 / 20
Tracking	MLflow — params, metrics, artifact
Retrain	daily 02:00 CET via pipeline

14 features, four families

Price history — current price, lag 1h, lag 12h, lag 24h, 6h rolling mean, 6h rolling volatility

Calendar — hour of day, day of week, is-weekend, is-holiday (broadcast join)

Weather — temperature, wind speed, humidity, solar radiation

Grid — actual load in MW

```
Shallow trees (depth 4) and min_child_weight 5
regularise deliberately — the dataset is
young and would overfit at default depth.
```

Why time-aware splitting matters here. A random split would put a zone's 18:00 price in training and its 17:00 lag feature in test — the model would be reading its own answer. Older records train, newer records test, so the evaluation reflects genuine forward prediction.

Latest production run — 530 scored predictions

72.3%
ACCURACY
383 of 530 correct

73.6%
PRECISION
alerts that were real

40.7%
RECALL
real spikes caught

0.524
F1 SCORE
harmonic mean

CONFUSION MATRIX · 27 ZONES

TRUE POSITIVE
81
spike called, spike happened

FALSE POSITIVE
29
spike called, none happened

FALSE NEGATIVE
118
spike missed

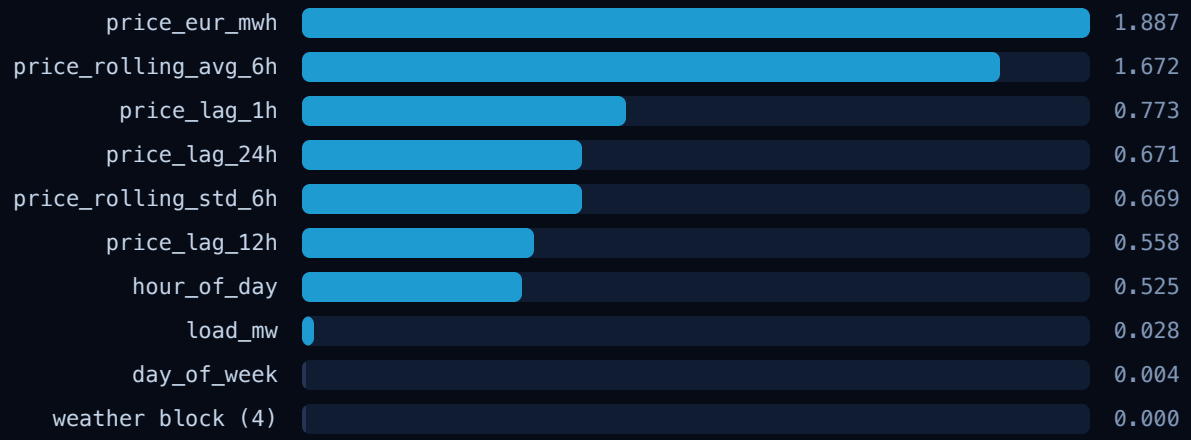
TRUE NEGATIVE
302
correctly quiet

The model is deliberately conservative. High precision with modest recall means it rarely cries wolf but does miss marginal spikes. For an alerting surface that is the right asymmetry — a false alarm costs an operator's attention, a missed marginal event costs very little.

Recall is a data-volume story. With two days of history, lag-12h and lag-24h are null for the first records in every zone, so the model is scoring partly blind on exactly the features that carry the signal. Recall rises as history accumulates — the architecture does not change.

What actually drives a spike prediction

Mean absolute SHAP across all 7,700 published explanations. Longer bar = the feature moves the spike probability further from its baseline.



Price history dominates

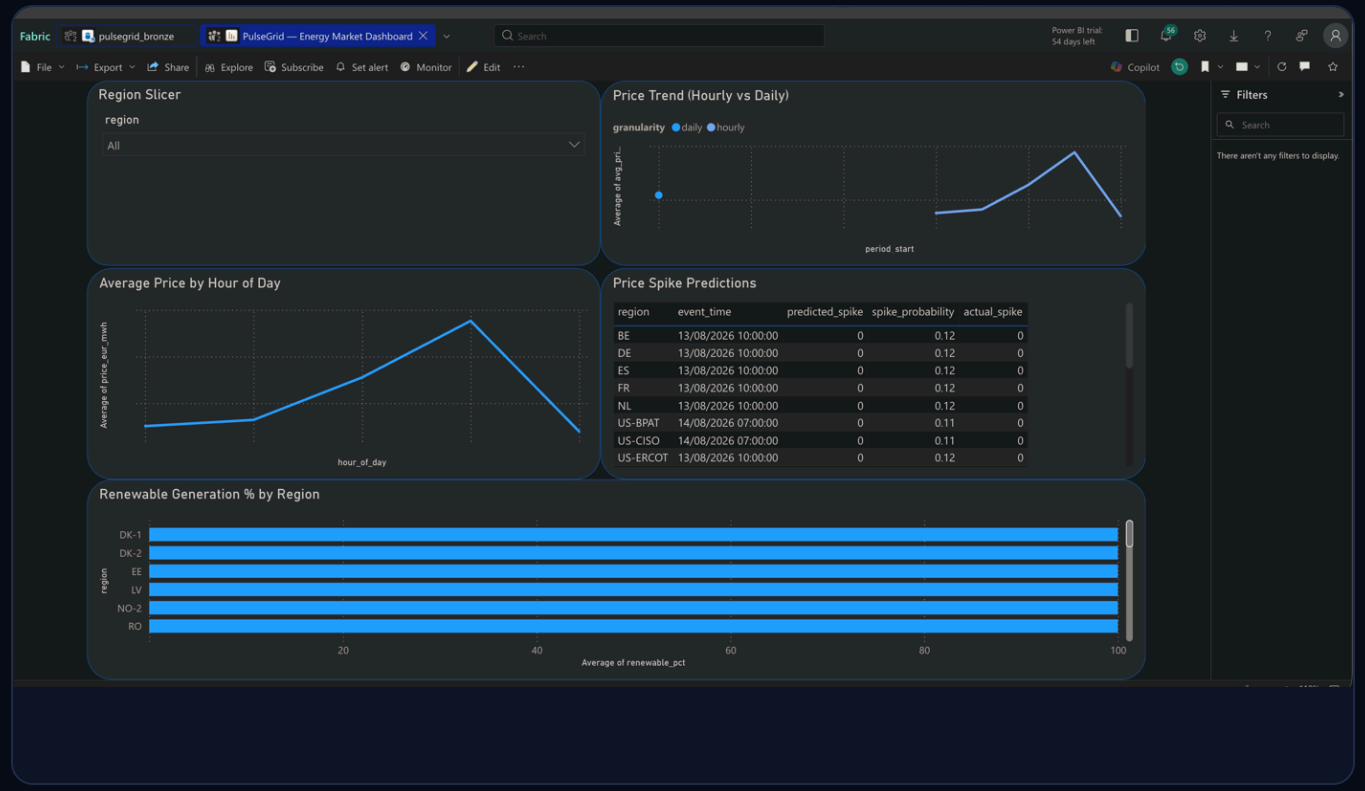
The current price and the 6-hour rolling average together carry most of the signal. Short lags and volatility follow. This matches how day-ahead markets actually behave — spikes are persistent, not isolated.

Weather contributes exactly zero — and that is a finding, not a bug

Temperature, wind, humidity and solar radiation all score 0.000. With two days of data there is no seasonal variance for the trees to split on, so those features are never selected. It is visible directly in the app's driver chart rather than hidden in a notebook.

SHAP is stored in long format — one row per (region, event_time, feature_name) with both the signed contribution and the feature value — so the AI analyst can answer "*why was DE-LU flagged at 18:00?*" with a single filter and sort.

Power BI semantic model over Direct Lake



Direct Lake — no import, no refresh

The model reads Gold Delta tables straight from OneLake. No data movement, no scheduled refresh, no duplicated storage. Gold tables were registered in the Lakehouse metastore with `saveAsTable` to make them addressable.

Five visuals, one page

Region slicer cross-filtering everything · average price by hour of day · hourly vs daily price trend · spike prediction table with probabilities · renewable share by region.

Relationships are region-only, on purpose

Aggregates use a truncated `period_start`; features use raw `event_time`. Joining mismatched time grains would produce silently wrong cross-filters, so time is handled per-visual instead.

The public dashboard — pulsegrid-energy.streamlit.app

Live snapshot · 29 zones priced · mean €144/MWh · 13 spike alerts of 40 scored

Six KPIs, six analytical panels

Zone price board ranked high → low and coloured by cross-zone percentile · multi-zone hourly price curve · spike watchlist · min–max spread per zone · stacked generation mix · SHAP driver chart filterable by zone.

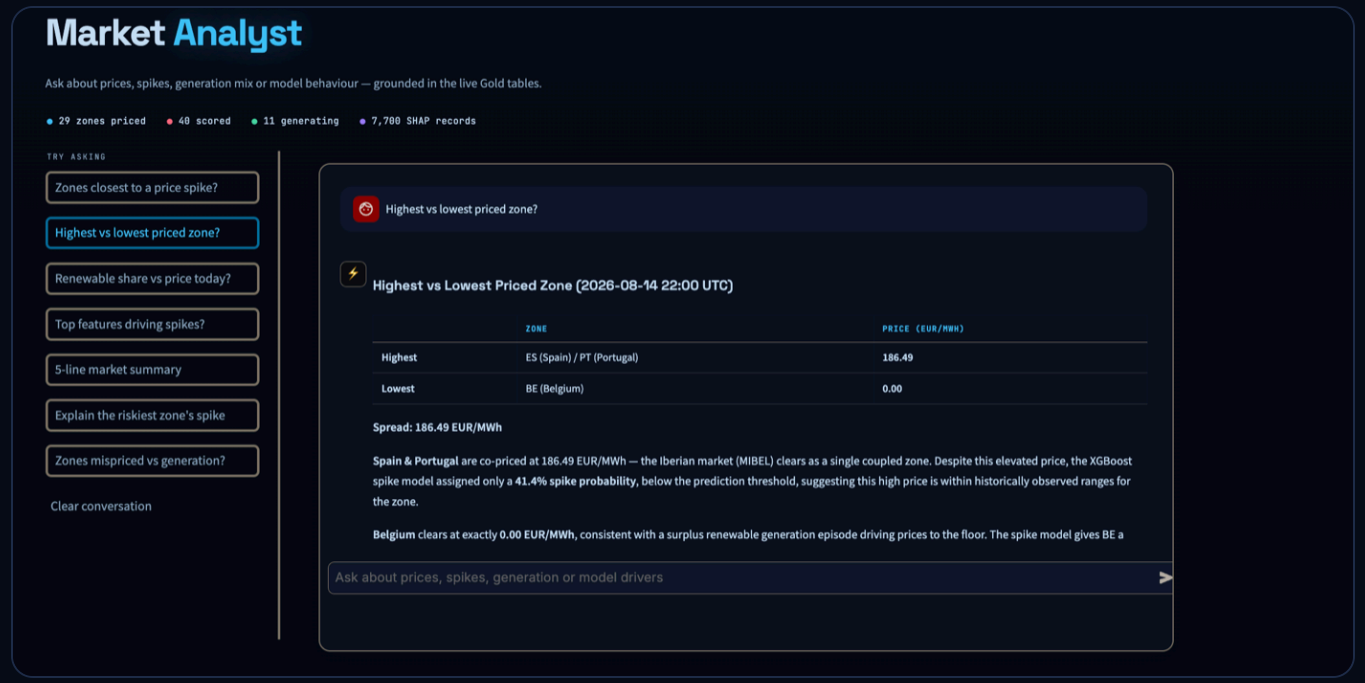
Zones reporting nothing are excluded, not drawn

ENTSO-E emits a row for every zone every interval; many TSOs report 0 MW across all fuels. Filtering those out shows 11 real generating zones instead of a wall of empty bars.

Every chart explains its own axes

A plain-English note sits under each chart — "taller and redder = more expensive right now" — so the page is readable by someone who has never traded power.

Ask the market a question in plain English



Streaming reply, grounded in the live Gold snapshot

Grounded, not generative

A compact context block — latest price per zone, spike predictions, generation mix and SHAP importance — is assembled from Gold and injected into the system prompt. Aggregates, not raw rows: the model needs market state, not 7,700 SHAP records.

Never invent a number

The prompt requires figures quoted from the snapshot with units. If the answer is not in the data, the model must name the Gold table that would hold it and when that table next refreshes — instead of guessing.

Degrades cleanly

Without an API key the chat shows a setup state and the dashboard keeps working. A failed request pops the user turn so the thread never desyncs.

Six pipelines on schedule, one alert watching the source

PIPELINE	SCHEDULE	LAYER
`pipeline_01a_daily_price`	daily 13:00 CET	Bronze
`pipeline_01b_realtime`	every 15 min	Bronze
`pipeline_01c_weather_eia`	every 30 min	Bronze
`pipeline_02_silver`	every 30 min	Silver
`pipeline_03_gold`	every 1 hour	Gold + publish
`pipeline_04_ml`	daily 02:00 CET	ML write-back

Max concurrency 1 on every schedule. Fabric Trial returns a 430 capacity error when Spark sessions overlap, so a slow run must never stack on the next tick. Failure email notification is enabled on all six.

Bronze freshness alert

A Fabric Activator rule runs every 30 minutes against the KQL database and emails if ingestion has gone quiet:

```
raw_electricity_prices
| where ingestion_time > ago(2h)
| count
| where Count == 0
```

Why this specific check. The worst failure in a real-time platform is not a crash — it is a poller that quietly stops writing while every dashboard keeps rendering the last good numbers. Pipeline failure emails cover runs that error; this rule covers runs that succeed and produce nothing.

Six real problems, and what was done about them

The 100 KB truncation bug

`mssparkutils.fs.head` is hard-capped at 100 KB regardless of its `maxBytes` argument. Three of four published tables were cut off mid-record, became invalid JSON, and the whole dashboard silently rendered zeros — only the one table under the cap displayed.

```
Fix: serialise in memory, push straight to the GitHub
contents API. No Lakehouse round trip, no cap.
```

Expiring Fabric tokens

Bearer tokens die after roughly an hour, which would break a deployed public app inside a single user session — and a service principal is not available on Trial.

```
Fix: publish Gold as a JSON snapshot committed to Git.
The deployed app never authenticates against anything.
```

%pip is disabled in pipeline runs

Notebooks that installed dependencies inline worked interactively and failed the moment a pipeline triggered them.

```
Fix: dependencies moved into pulsegrid_env public libraries,
pinned for Python 3.10 — xgboost 3.3+ requires 3.12.
```

Blocked weather API

Fabric Trial Spark executors block outbound HTTP to Open-Meteo entirely — the failure looked like a timeout, not a policy block.

```
Fix: connectivity probe cell to diagnose, then Visual Crossing
as a confirmed-reachable replacement inside its free tier.
```

Duplicate columns breaking joins

Bronze carries `load_mw` and `temperature_c` on the price table; joining the Silver versions raised `AnalysisException` on ambiguous references.

```
Fix: explicit drop() of the Bronze copies before every join,
so the Silver column is unambiguously the one that survives.
```

Trial capacity 430 errors

Running pollers simultaneously exhausted the Trial Spark capacity and every notebook failed together — the noisiest possible failure mode.

```
Fix: separate pipelines, staggered cadences, max concurrency 1
on every schedule.
```

What the platform delivers today

Measured from the Gold snapshot published 14 August 2026, 22:00 UTC.

29	144€	186€	13	80%	7,700
ZONES PRICED	MEAN PRICE	PEAK ZONE	SPIKE ALERTS	RENEWABLE SHARE	SHAP RECORDS
of 40 tracked	EUR/MWh, latest hour	Spain	of 40 zones scored	11 generating zones	model explanations

A platform that runs itself

Six scheduled pipelines move data from four public APIs through Bronze, Silver and Gold, retrain the classifier nightly, publish a fresh snapshot hourly, and page on ingestion silence — with no manual step anywhere in the loop.

A model you can interrogate

72.3% accuracy at 73.6% precision on 530 live predictions, with every one of them decomposed into 14 signed SHAP contributions stored in the lakehouse and queryable from both the dashboard and the chat.

Two surfaces, two audiences

A Direct Lake Power BI model for the analyst inside the tenant, and a public Streamlit app with a Claude analyst for everyone else — both reading the same Gold contract.

Scope of the numbers. The platform has been running on live data for a short window, so the price and spike figures are a genuine snapshot rather than a long-run benchmark. Recall and the weather features' contribution both improve directly with accumulated history — the pipeline, the model and the serving layer do not change.

THANK YOU

Ingest, refine, predict, explain, serve — on one platform, end to end.

Live application

pulsegrid-energy.streamlit.app

Dashboard + Claude analyst.
No login required.

Source code

github.com/demonjd2026-afk/
pulsegrid-fabric-realtime

6 notebooks, full Streamlit app,
README + TECHSPEC.

Author

Jayanth Dolai
Senior Data Engineer
Azure · Databricks · Microsoft Fabric

Databricks DE Associate · DP-900
DP-700 · DP-600 · GenAI Associate